MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 27 — Bases de Données avec Python

SQLite, SQLAlchemy, PostgreSQL — persister les données proprement

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

25 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 27 — Bases de Données avec Python **2**
0.1 SQLite — La base de données embarquée 2
0.2 CRUD — Créer, Lire, Mettre à jour, Supprimer 3
0.3 SQL avancé — Jointures et agrégations 5
0.4 SQLAlchemy — L'ORM Python 6
0.5 Gestionnaire de contexte et transactions 9
0.6 Ce qui m'a le plus surpris dans ce chapitre 11
0.7 Résumé du Chapitre 27 11

Chapitre 27 — Bases de Données avec Python

À retenir

Une **base de données** permet de stocker, organiser et interroger des données de façon structurée et persistante. Python offre plusieurs approches : **SQLite** pour une base légère embarquée, **SQLAlchemy** pour une abstraction orientée objet des bases relationnelles, et des connecteurs pour **PostgreSQL** et **MySQL** en production. Ce chapitre couvre le SQL depuis Python, l'ORM, et les bonnes pratiques de persistance de données.

SQLite — La base de données embarquée

sqlite3 — connexion et création de table

```python
import sqlite3
from pathlib import Path

# Connexion      cree le fichier si inexistant
conn = sqlite3.connect("ecole.db")

# Activer les cles etrangeres
conn.execute("PRAGMA foreign_keys = ON")

# Retourner des dicts plutot que des tuples
conn.row_factory = sqlite3.Row

cursor = conn.cursor()

# Creer les tables
cursor.executescript("""
    CREATE TABLE IF NOT EXISTS etudiants (
        id        INTEGER PRIMARY KEY AUTOINCREMENT,
        prenom    TEXT    NOT NULL,
        nom       TEXT    NOT NULL,
        email     TEXT    UNIQUE NOT NULL,
        ville     TEXT,
        created   TEXT    DEFAULT (date('now'))
    );

    CREATE TABLE IF NOT EXISTS notes (
        id           INTEGER PRIMARY KEY AUTOINCREMENT,
        etudiant_id  INTEGER NOT NULL,
        matiere      TEXT    NOT NULL,
        note         REAL    CHECK(note >= 0 AND note <= 20),
        date_exam    TEXT,
```

```
        FOREIGN KEY (etudiant_id)
            REFERENCES etudiants(id)
            ON DELETE CASCADE
    );

    CREATE INDEX IF NOT EXISTS idx_etudiant_email
        ON etudiants(email);
    CREATE INDEX IF NOT EXISTS idx_notes_etudiant
        ON notes(etudiant_id);
""")

conn.commit()
print("Tables creees avec succes.")
conn.close()
```

CRUD — Créer, Lire, Mettre à jour, Supprimer

SQLite — opérations CRUD complètes

```
import sqlite3

# Utiliser with pour fermeture automatique
with sqlite3.connect("ecole.db") as conn:
    conn.row_factory = sqlite3.Row
    cur = conn.cursor()

    # CREATE     inserer des donnees
    # Toujours utiliser des parametres (?)     jamais de f-
       string !
    etudiants = [
        ("Alice",   "Dupont",  "alice@ensea.ci",  "Abidjan"),
        ("Eddye",     "Julio",  "eddye@ensea.ci",    "Lome"),
        ("Dagnon", "Mariam",    "dagnon@ensea.ci","Accra"),
        ("Marie",   "Corine",  "marie@ensea.ci",  "Dakar"),
        ("Eve",     "Traore",  "eve@ensea.ci",    "Bamako"),
    ]
    cur.executemany(
        "INSERT OR IGNORE INTO etudiants "
        "(prenom, nom, email, ville) VALUES (?,?,?,?)",
        etudiants
    )

    # Inserer des notes
    notes = [
        (1, "Python",   15.5, "2026-03-15"),
        (1, "Stats",    17.0, "2026-03-16"),
        (2, "Python",   18.0, "2026-03-15"),
        (2, "Stats",    14.5, "2026-03-16"),
```

```
        (3, "Python",   12.0, "2026-03-15"),
        (4, "Python",    9.5, "2026-03-15"),
        (5, "Python",   16.0, "2026-03-15"),
        (5, "Stats",    15.5, "2026-03-16"),
    ]
    cur.executemany(
        "INSERT INTO notes "
        "(etudiant_id, matiere, note, date_exam) "
        "VALUES (?,?,?,?)",
        notes
    )
    conn.commit()

    # READ     lire des donn es
    cur.execute("SELECT * FROM etudiants")
    for row in cur.fetchall():
        print(f"  {row['prenom']:10} {row['nom']:10} "
              f"| {row['ville']}")

    # Requete avec filtre
    cur.execute(
        "SELECT * FROM etudiants WHERE ville = ?",
        ("Lome",)
    )
    print(f"\n Etudiants de Lome : {cur.fetchall()}")

    # UPDATE     mettre    jour
    cur.execute(
        "UPDATE etudiants SET ville = ? WHERE email = ?",
        ("Grand-Bassam", "alice@ensea.ci")
    )
    print(f"\nLignes modifiees : {cur.rowcount}")

    # DELETE     supprimer
    cur.execute(
        "DELETE FROM notes WHERE note < ? AND matiere = ?",
        (10.0, "Python")
    )
    print(f"Notes supprimees : {cur.rowcount}")
    conn.commit()
```

SQL avancé — Jointures et agrégations

Requêtes SQL complexes depuis Python

```python
import sqlite3
import pandas as pd

with sqlite3.connect("ecole.db") as conn:
    conn.row_factory = sqlite3.Row

    # JOIN      combiner deux tables
    requete_join = """
        SELECT
            e.prenom,
            e.nom,
            e.ville,
            n.matiere,
            n.note,
            CASE
                WHEN n.note >= 16 THEN 'TB'
                WHEN n.note >= 14 THEN 'B'
                WHEN n.note >= 12 THEN 'AB'
                WHEN n.note >= 10 THEN 'P'
                ELSE 'F'
            END AS mention
        FROM etudiants e
        INNER JOIN notes n ON e.id = n.etudiant_id
        ORDER BY n.note DESC
    """
    cur = conn.cursor()
    cur.execute(requete_join)
    print(f"{'Etudiant':<15} {'Matiere':<10} "
          f"{'Note':>5}  {'Mention'}")
    print("-" * 45)
    for row in cur.fetchall():
        print(f"  {row['prenom']:<13} {row['matiere']:<10} "
              f"{row['note']:>5.1f}  {row['mention']}")

    # GROUP BY      statistiques par matiere
    requete_stats = """
        SELECT
            n.matiere,
            COUNT(*)            AS nb_etudiants,
            ROUND(AVG(n.note),2) AS moyenne,
            MIN(n.note)         AS min_note,
            MAX(n.note)         AS max_note,
            SUM(CASE WHEN n.note >= 10 THEN 1 ELSE 0 END)
                                AS nb_admis
        FROM notes n
        GROUP BY n.matiere
```

```
        ORDER BY moyenne DESC
    """
    cur.execute(requete_stats)
    print(f"\n{'Matiere':<12} {'N':>3} {'Moy':>6} "
          f"{'Min':>5} {'Max':>5} {'Admis':>6}")
    print("-" * 45)
    for row in cur.fetchall():
        print(f"  {row['matiere']:<10} {row['nb_etudiants']:>3} "
              f"{row['moyenne']:>6.2f} {row['min_note']:>5.1f} "
              f"{row['max_note']:>5.1f} {row['nb_admis']:>6}")

    # Charger directement dans Pandas !
    df = pd.read_sql_query(requete_join, conn)
    print(f"\nDataFrame : {df.shape}")
    print(df.describe())
```

SQLAlchemy — L'ORM Python

SQLAlchemy ORM — définir les modèles

```
from sqlalchemy import (create_engine, Column, Integer,
                        String, Float, ForeignKey, DateTime,
                        CheckConstraint, Index)
from sqlalchemy.orm import (declarative_base, relationship,
                            Session, joinedload)
from sqlalchemy.sql import func
from datetime import datetime

# Moteur      SQLite ici, PostgreSQL en prod
engine = create_engine(
    "sqlite:///ecole_orm.db",
    echo=False    # True pour voir le SQL genere
)

Base = declarative_base()

# ============================================================
# MODELES      classes Python = tables SQL
# ============================================================

class Etudiant(Base):
    __tablename__ = "etudiants"

    id       = Column(Integer, primary_key=True)
    prenom   = Column(String(50), nullable=False)
    nom      = Column(String(50), nullable=False)
```

```python
    email   = Column(String(100), unique=True, nullable=False)
    ville   = Column(String(50))
    created = Column(DateTime, default=datetime.utcnow)

    # Relation     acces direct aux notes
    notes   = relationship("Note", back_populates="etudiant",
                           cascade="all, delete-orphan")

    @property
    def moyenne(self):
        if not self.notes:
            return None
        return sum(n.note for n in self.notes) / len(self.notes
           )

    def __repr__(self):
        return f"<Etudiant {self.prenom} {self.nom}>"


class Note(Base):
    __tablename__ = "notes"
    __table_args__ = (
        CheckConstraint("note >= 0 AND note <= 20"),
    )

    id          = Column(Integer, primary_key=True)
    etudiant_id = Column(Integer,
                         ForeignKey("etudiants.id"),
                         nullable=False)
    matiere     = Column(String(50), nullable=False)
    note        = Column(Float, nullable=False)
    date_exam   = Column(DateTime, default=datetime.utcnow)

    etudiant = relationship("Etudiant",
                            back_populates="notes")

    @property
    def mention(self):
        if self.note >= 16: return "TB"
        if self.note >= 14: return "B"
        if self.note >= 12: return "AB"
        if self.note >= 10: return "P"
        return "F"

    def __repr__(self):
        return f"<Note {self.matiere}:{self.note}>"


# Creer toutes les tables
Base.metadata.create_all(engine)
```

```python
print("Tables ORM creees.")
```

SQLAlchemy ORM — opérations CRUD

```python
from sqlalchemy.orm import Session

# Session      unite de travail
with Session(engine) as session:

    # CREATE      ajouter des objets Python
    alice = Etudiant(prenom="Alice", nom="Dupont",
                     email="alice@orm.ci", ville="Abidjan")
    bob   = Etudiant(prenom="Eddye",   nom="Julio",
                     email="eddye@orm.ci",   ville="Lome")

    session.add_all([alice, bob])
    session.flush()    # obtenir les IDs sans commit

    # Ajouter des notes via la relation
    alice.notes.append(Note(matiere="Python", note=15.5))
    alice.notes.append(Note(matiere="Stats",  note=17.0))
    eddye.notes.append(Note(matiere="Python",   note=18.0))
    eddye.notes.append(Note(matiere="Stats",    note=14.5))

    session.commit()
    print(f"Alice ID : {alice.id}")
    print(f"Moyenne Alice : {alice.moyenne:.1f}")

    # READ      requetes ORM
    tous = session.query(Etudiant).all()
    for e in tous:
        print(f"  {e.prenom}     moy: {e.moyenne:.1f}")

    # Filtre
    abidjanais = (session.query(Etudiant)
                  .filter(Etudiant.ville == "Abidjan")
                  .all())

    # Jointure eager loading
    etudiants_notes = (
        session.query(Etudiant)
        .options(joinedload(Etudiant.notes))
        .all()
    )
    for e in etudiants_notes:
        for n in e.notes:
            print(f"  {e.prenom} | {n.matiere} : "
                  f"{n.note} ({n.mention})")
```

```python
    # UPDATE
    alice.ville = "Grand-Bassam"
    session.commit()

    # DELETE
    mauvaises = (session.query(Note)
                 .filter(Note.note < 10)
                 .all())
    for n in mauvaises:
        session.delete(n)
    session.commit()
    print(f"Notes supprimees : {len(mauvaises)}")
```

Gestionnaire de contexte et transactions

Transactions — atomicité des opérations

```python
import sqlite3
from contextlib import contextmanager

@contextmanager
def connexion_db(chemin="ecole.db"):
    """Gestionnaire de contexte pour la BDD."""
    conn = sqlite3.connect(chemin)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    try:
        yield conn
        conn.commit()
    except Exception as e:
        conn.rollback()   # annuler si erreur !
        print(f"Transaction annulee : {e}")
        raise
    finally:
        conn.close()


def transferer_etudiant(id_etudiant, nouvelle_ville):
    """Exemple de transaction atomique."""
    with connexion_db() as conn:
        cur = conn.cursor()

        # Verifier que l'etudiant existe
        cur.execute(
            "SELECT id, prenom FROM etudiants WHERE id=?",
            (id_etudiant,)
        )
        etudiant = cur.fetchone()
```

```
        if not etudiant:
            raise ValueError(f" tudiant  {id_etudiant} inconnu"
                )

        # Mettre a jour
        cur.execute(
            "UPDATE etudiants SET ville=? WHERE id=?",
            (nouvelle_ville, id_etudiant)
        )

        # Logger le changement (operation liee)
        cur.execute(
            "INSERT INTO logs (action, detail, date) "
            "VALUES (?, ?, date('now'))",
            ("TRANSFERT",
             f"{etudiant['prenom']}    {nouvelle_ville}")
        )
        # Si l'une echoue, les deux sont annulees

    print(f"Transfert effectue.")


# Migrations simples
def migrer_db(conn, version_actuelle):
    """Appliquer des migrations de schema."""
    migrations = {
        1: "ALTER TABLE etudiants ADD COLUMN telephone TEXT",
        2: "ALTER TABLE notes ADD COLUMN coefficient REAL
           DEFAULT 1.0",
        3: "CREATE INDEX IF NOT EXISTS idx_notes_matiere ON
           notes(matiere)",
    }
    cur = conn.cursor()
    for version, sql in migrations.items():
        if version > version_actuelle:
            try:
                cur.execute(sql)
                print(f"Migration {version} appliquee.")
            except sqlite3.OperationalError as e:
                print(f"Migration {version} ignoree : {e}")
    conn.commit()
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Les paramètres ? ne sont pas optionnels — ils sont vitaux. Écrire `f"SELECT * FROM users WHERE email = 'email'"` semble pratique. C'est une **injection SQL** — la vulnérabilité la plus exploitée du web. Un utilisateur malveillant peut saisir `' OR '1'='1` et accéder à toute la base. Les paramètres `?` de SQLite ou `:param` de SQLAlchemy échappent automatiquement les entrées. C'est une règle absolue, sans exception.

2. L'ORM pense en objets, pas en tables. Avec SQLAlchemy ORM, je n'écris plus `SELECT * FROM notes WHERE etudiant_id = 1`. J'écris `alice.notes` et j'obtiens directement la liste des objets `Note` d'Alice. Cette traduction transparente entre le monde objet Python et le monde relationnel SQL m'a révélé ce que signifie vraiment "abstraction" — cacher la complexité sans perdre la puissance.

3. `pd.read_sql_query()` connecte SQL et Pandas. Pouvoir écrire une requête SQL complexe avec jointures et agrégations, puis charger le résultat directement dans un DataFrame Pandas pour visualisation ou ML — en une seule ligne — est l'une des intégrations les plus puissantes de l'écosystème Python Data Science. SQL pour extraire, Pandas pour analyser, Matplotlib pour visualiser : le pipeline est complet.

Résumé du Chapitre 27

Ce que j'ai appris dans ce chapitre :

- **SQLite** : base embarquée, zéro configuration, parfaite pour apprendre.
- **CRUD** : `INSERT`, `SELECT`, `UPDATE`, `DELETE` depuis Python.
- Paramètres `?` — protection absolue contre l'injection SQL.
- `JOIN`, `GROUP BY`, `CASE WHEN` — SQL avancé depuis Python.
- `pd.read_sql_query()` — charger un résultat SQL dans Pandas.
- **SQLAlchemy ORM** : classes Python = tables SQL, relations automatiques.
- `Session`, `add()`, `query()`, `commit()` — cycle de vie ORM.
- **Transactions** : `commit()` / `rollback()` — atomicité garantie.
- Gestionnaire de contexte BDD — fermeture et rollback automatiques.

"Une base de données bien conçue, c'est la mémoire organisée d'une application. SQL est le langage de cette mémoire — apprendre à l'écrire proprement, c'est apprendre à penser en données."